



KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science and Engineering

Natural Language Processing Laboratory

Course Code: CSE 4122

Project Proposal

Fine-Grained Classification of Complaints in Google Play Store Reviews Using Transformers

Complaint-Category Identification in Mobile Game Reviews using a Fine-Tuned Transformer Classifier

Submitted to,

Dr. K. M. Azharul Hasan

Professor

Department of Computer Science and Engineering

Mr. Md. Nazirulhasan Shawon

Assistant Professor

Department of Computer Science and Engineering

Submitted by,

Arka Braja Prasad Nath

Roll: 2107055

Farhan Tahmid

Roll: 2107059

Section: A

Year: 4th

Semester: 1st

Submission Date: 19/08/2026

1 Introduction Section

1.1 Introduction

Google Play Store reviews contain large amounts of direct user feedback about mobile applications and games. A simple positive/negative sentiment label is often not enough to explain what users are actually dissatisfied with. For example, a review such as “The game freezes after every match and shows too many ads” contains at least two separate complaint types: performance problems and excessive advertising. This project therefore focuses on identifying the specific negative issues expressed in reviews rather than only assigning a general sentiment label.

1.2 Problem Statement

Most general review-analysis systems reduce user feedback to sentiment or broad categories, which can hide multiple concrete problems mentioned in the same review. A single negative review may simultaneously complain about lag, crashes, misleading gameplay, excessive advertisements, pay-to-win mechanics, or payment-related issues. The problem addressed in this project is to automatically assign one or more fine-grained complaint labels to each negative Google Play Store review.

1.3 Objective

The objective is to build a transformer-based classifier that takes a negative Google Play Store review as input and predicts all relevant complaint categories present in that review. The emphasis is on producing useful fine-grained labels rather than comparing multiple model families. For example, “The app is very slow and keeps crashing” should be labeled with both `PERFORMANCE/LAG` and `CRASH/FREEZE`.

1.4 NLP Task Type

The project is a **multi-label text classification** task. Unlike multiclass classification, where only one class is selected, multi-label classification allows a review to receive several labels at the same time. For instance, “Too many ads and you cannot progress without paying” may receive both `EXCESSIVE_ADS` and `PAY_TO_WIN`.

1.5 Task Boundary

The main classifier will operate only on negative reviews. During data collection, a separate existing positive/negative review detector will be used to filter the collected reviews, but that detector is outside the scope of this project and will not be trained or evaluated as part of the proposed system. For example, “Great game, works perfectly” would be removed before the complaint-classification stage.

1.6 Target App Population

The project will focus on **mobile games** available on the Google Play Store. Restricting the domain to games makes the complaint taxonomy more coherent because issues such as lag, crashes, deceptive gameplay, excessive advertisements, pay-to-win mechanics, missing rewards, and aggressive monetization are all common and meaningful within the same application category.

1.7 Language Scope

The initial dataset will contain **English-language reviews only**. This reduces annotation ambiguity and allows the use of established English transformer models such as BERT, DistilBERT,

or RoBERTa without introducing multilingual modeling as a separate research problem.

1.8 Related Work

Related work will cover Google Play Store review analysis, sentiment analysis of mobile-app reviews, aspect-based sentiment analysis, complaint classification, and fine-grained or multi-label review classification. The project will specifically examine how previous studies categorize user complaints and identify where existing taxonomies differ from the proposed game-review complaint labels. Table 1 summarizes three works that directly motivate the taxonomy design, the classification framing, and the transformer-based modeling approach adopted in this project.

Table 1: Related work informing the proposed fine-grained complaint classification approach.

Research paper	Main contribution	Relevance to this project
Guzman and Maalej, “How Do Users Like This Feature? A Fine-Grained Sentiment Analysis of App Reviews” [1]	Mines candidate app features from review text via collocation extraction and computes a separate sentiment score for each identified feature.	Motivates treating a single review as a source of several distinct, concrete issues rather than one overall sentiment label, supporting the project’s fine-grained taxonomy.
Maalej and Nabil, “Bug Report, Feature Request, or Simply Praise? On Automatically Classifying App Reviews” [2]	Proposes probabilistic (Naive Bayes, Maximum Entropy) and rule-based techniques to sort app reviews into bug report, feature request, user experience, and rating categories.	Provides a foundational single-label review-classification scheme; the project’s complaint taxonomy extends this idea into a finer-grained, multi-label setting where several categories can co-occur.
Hadi and Fard, “Evaluating Pre-Trained Models for User Feedback Analysis in Software Engineering: A Study on Classification of App-Reviews” [3]	Benchmarks BERT, RoBERTa, DistilBERT, and ALBERT on app-review classification tasks and reports consistent gains for transformer models over classical machine-learning baselines.	Directly supports the choice of a single fine-tuned transformer (BERT, DistilBERT, or RoBERTa) as the classifier backbone rather than a non-transformer baseline.

1.9 Label Taxonomy

The current plan is to use a **fine-grained flat taxonomy** of approximately 10–15 complaint labels. Each label should represent a concrete and observable issue rather than a vague judgment such as “bad game”.

Possible labels may include:

- PERFORMANCE/LAG
- CRASH/FREEZE
- EXCESSIVE_ADS
- DECEPTIVE_ADS

- MISLEADING_GAMEPLAY/CONTENT
- PAY_TO_WIN
- AGGRESSIVE_MONETIZATION
- SUBSCRIPTION/PAYMENT_ISSUE
- REWARD_NOT_RECEIVED
- LOGIN/ACCOUNT_ISSUE
- UI/UX_PROBLEM
- MISSING/BROKEN_FEATURE

The final number of labels is still undecided. Two possible choices are:

- **Fine-grained taxonomy:** approximately 10–15 labels for more detailed complaint identification.
- **Broader taxonomy:** approximately 6–8 labels if the collected data is too sparse or several labels strongly overlap.

1.10 Annotation Unit

The annotation unit will be the **entire review**. Each review will receive a binary value for every label. For example, “The game is laggy and crashes whenever an ad appears” may receive PERFORMANCE/LAG = 1, CRASH/FREEZE = 1, and EXCESSIVE_ADS = 1.

2 Data Handling Section

2.1 Data Collection Plan and Sampling Strategy

Reviews will be collected from selected Google Play Store games. The primary sampling method will be **random sampling** so that the dataset reflects the natural distribution of complaints and does not become dependent on manually chosen keywords. If important complaint labels are too rare, limited keyword-based sampling may be used only to enrich those specific classes. For example, terms such as “refund” or “reward” may be used to find additional examples of rare payment or reward-related complaints after the random sample has been analyzed.

2.2 Cleaning / Preprocessing

Only minimal preprocessing will be applied because transformer models benefit from retaining most of the original textual context. URLs, obvious usernames or identifiers, duplicate whitespace, malformed characters, and exact duplicate reviews will be removed where necessary. Negation, punctuation, spelling variation, monetary values, and complaint-related terms will be preserved because they may affect meaning. For example, “not laggy anymore” should not be reduced to “laggy” through aggressive preprocessing.

2.3 Tokenization

The native tokenizer corresponding to the selected transformer model will be used. For example, BERT will use its WordPiece tokenizer and RoBERTa will use its own subword tokenizer. A custom tokenizer will not be trained because the pretrained model and tokenizer are designed to operate together.

2.4 Text Representation

The standard sequence representation produced by the selected transformer will be used for classification. For BERT-style models, this typically means using the final representation associated with the classification token and passing it to a linear classification layer. No separate manually trained embedding model will be required.

2.5 Class Distribution, Class Imbalance, and No-Target Case

The number of samples for every label will be analyzed before training. If substantial imbalance exists, **weighted Binary Cross-Entropy loss** will be used so that rare complaint labels contribute more appropriately to the training objective. If the distribution is sufficiently balanced, standard Binary Cross-Entropy will be used instead.

A negative review that does not match any defined target complaint will be represented using an **all-zero label vector** rather than forcing it into an artificial **OTHER** class. For example, if “The storyline is boring” does not correspond to any selected label, all target labels may remain 0.

2.6 Train / Validation / Test Split Strategy

A random review-level split will be used for the dataset, with separate training, validation, and test sets. A typical split such as 70% training, 15% validation, and 15% testing may be used. The split should preserve the multi-label class distribution as much as possible so that rare complaint types do not disappear from the validation or test sets.

3 Train–Validation–Test Section

3.1 Transformer Model Selection

The final transformer model has not yet been selected. The current options are:

- **DistilBERT** – smaller and faster, suitable when computational resources are limited.
- **BERT-base** – a standard and well-understood transformer baseline for text classification.
- **RoBERTa-base** – a strong general-purpose transformer that may provide better classification performance while remaining manageable for an undergraduate project.

Only one transformer model is intended for the final project; the work is not designed as a baseline-versus-transformer comparison study.

3.2 Training Methodology

3.2.1 Fine-Tuning

The selected pretrained transformer will be **fully fine-tuned** on the labeled review dataset rather than training only the final classification head. This allows the internal contextual representations to adapt to game-review language and complaint patterns.

3.2.2 Multi-Label Output Architecture

If the taxonomy contains K labels, the final classification layer will produce K logits:

$$z_1, z_2, \dots, z_K.$$

Each logit will be passed independently through a sigmoid function so that several labels can be predicted at the same time. For example, one review may obtain high probabilities for both CRASH/FREEZE and EXCESSIVE_ADS.

3.2.3 Loss Function

The primary loss function will be **Binary Cross-Entropy with logits**. If class-distribution analysis shows substantial imbalance, a weighted version of Binary Cross-Entropy will be used. The decision between standard and weighted loss will therefore be made after examining the final label frequencies.

3.2.4 Decision-Threshold Strategy

Instead of using a single fixed threshold such as 0.5 for every complaint label, a **separate threshold for each label** will be selected using only the validation set. This is useful because some labels may require a lower or higher probability threshold to achieve a reasonable balance between precision and recall.

3.2.5 Training Stop Criterion

Training will use early stopping based on **validation Macro-F1**. Training will stop when Macro-F1 no longer improves for a predefined number of validation checks or epochs. Macro-F1 is used because every complaint category should contribute equally, including rare labels.

3.3 Testing and Evaluation Metrics

The final model will be evaluated only on the held-out test set after all model settings and thresholds have been finalized using the training and validation sets. Evaluation will include **Macro-F1, Micro-F1, Precision, Recall, and per-label confusion matrices**. Macro-F1 will show whether the model performs consistently across both common and rare complaint categories, while Micro-F1 will summarize overall multi-label performance. Per-label precision, recall, F1, and confusion analysis will be used to identify which complaint types are easy or difficult for the classifier.

For example, if PAY_TO_WIN has high recall but low precision, the model may be incorrectly assigning that label to general monetization complaints. Such cases will be examined during the final error analysis.

4 Conclusion

This proposal frames negative Google Play Store review analysis as fine-grained multi-label classification rather than single-label sentiment scoring, since a single complaint often bundles several distinct issues such as lag, crashes, excessive ads, and pay-to-win mechanics. Restricting the scope to English-language mobile game reviews keeps the complaint taxonomy coherent and lets the project rely on established pretrained transformers (DistilBERT, BERT-base, or RoBERTa-base) without introducing multilingual modeling as a separate problem. A single transformer will be fully fine-tuned with an independent sigmoid output per label, trained with (weighted) binary cross-entropy, and evaluated using per-label decision thresholds selected on the validation set, so that both frequent and rare complaint categories are represented fairly in the final Macro-F1, Micro-F1, precision, recall, and confusion-matrix analysis. The related work in Table 1 shows that fine-grained, multi-label framing and transformer-based classifiers are well established directions in app-review analysis, and this project applies that combination specifically to the under-studied case of negative mobile-game reviews. The expected outcome is a classifier that gives game developers actionable, per-issue visibility into what is actually driving negative feedback, rather than a single aggregate negative rating.

References

- [1] E. Guzman and W. Maalej, “How Do Users Like This Feature? A Fine-Grained Sentiment Analysis of App Reviews,” in *Proceedings of the 22nd IEEE International Requirements Engineering Conference (RE)*, 2014, pp. 153–162. https://www.researchgate.net/publication/282272480_How_Do_Users_Like_This_Feature_A_Fine_Grained_Sentiment_Analysis_of_App_Reviews
- [2] W. Maalej and H. Nabil, “Bug Report, Feature Request, or Simply Praise? On Automatically Classifying App Reviews,” in *Proceedings of the 23rd IEEE International Requirements Engineering Conference (RE)*, 2015, pp. 116–125. https://www.researchgate.net/publication/301442885_Bug_report_feature_request_or_simply_praise_On_automatically_classifying_app_reviews
- [3] M. A. Hadi and F. H. Fard, “Evaluating Pre-Trained Models for User Feedback Analysis in Software Engineering: A Study on Classification of App-Reviews,” *arXiv preprint arXiv:2104.05861*, 2022. <https://arxiv.org/abs/2104.05861>